

Design and FPGA Implementation of a Viterbi Decoder: A Case Study Using SystemVerilog and Co-Simulation

Jiuling Tang
Dept. of Electrical Engineering
University of Windsor
Windsor, ON, N9B 3P4, Canada
{jtang.email@yahoo.com}

Abstract – Design and field programmable gate array (FPGA) implementation of a high-speed Viterbi decoder with a constraint length of 7 and a code rate of $\frac{1}{2}$ is presented here. Based on an Algorithmic State Machine (ASM) flowchart of Viterbi algorithm and by taking advantages of lookup tables and parallelism, this hard-decision decoder has a throughput of 510 Mbps. Its RTL code and self-checking test bench are all written in SystemVerilog. The code is functionally verified in various situations, including several directed test cases and numerous random test cases. It is also verified in the Simulink and ModelSim co-simulation environment. The FPGA implementation results show that its core can run as high as 510 MHz, and has 80.39 ns latency.

Keywords - Convolutional encoder, Viterbi decoder, FPGA, SystemVerilog, Co-simulation.

I. INTRODUCTION

The Viterbi algorithm, proposed in 1967 [1], is the most extensively employed decoding algorithm for convolutional codes. It is a very powerful method for forward error correction. It has been widely deployed in many wireless communication systems, like IEEE 802.11a/g, WiMax, DAB/DVB, WCDMA and GSM, to improve the capacity of communication channels.

The consumer demands for sophisticated portable wireless communication devices are high. So the needs for high speed Viterbi decoder are increasing. A number of papers have reported the speed improvement by using different Viterbi decoder realizations, like parallel Viterbi algorithms and look-ahead algorithms. Some of the designs are implemented as ASICs, and others as FPGAs. Some of them are hard-decision Viterbi decoders, and some of them are soft-decision ones. The clock speed of their FPGA implementations ranges from 40 MHz [2] to 163 MHz [3].

This paper presents design and verification of a high-speed Viterbi decoder by using SystemVerilog and .Modelsim/Simulink co-simulation. The decoder FPGA implementation shows that its core can run as high as 510 MHz with a throughput of 510Mbps.

The rest of the paper is organized as follows: Section II describes some background of Viterbi decoding. Section III is about the design of the proposed Viterbi decoder. Section IV presents the verification of the design and the techniques used. Section V is the FPGA implementation. Section VI concludes the study.

II. BACKGROUND OF VITERBI DECODING

In communication systems, forward error correction (FEC) is an error control method for data transmission by adding redundant data to its messages to improve the capacity of a channel. This redundant data allows the receiver to detect and to correct a certain number of errors without asking the sender to re-transmit more additional data. The process of adding this redundant information is known as channel coding.

Mainly, there are two major categories of channel coding algorithms [4]: block coding, like Reed-Solomon coding, and convolutional coding. Block codes operate on relatively large message blocks, whereas convolutional codes operate on serial data, one or a few bits at a time. There are a variety of convolutional codes and decoding algorithms.

A convolutional encoder takes k input bits and maps them into n output bits according to its generator polynomials. The output bits depend on both current k information bits and previous ones. Figure 1 illustrates a block diagram of a convolutional encoder (2, 1, 7).

There are basically three generic methods for decoding convolutional codes: feedback decoding, sequential decoding, and maximum likelihood decoding. The Viterbi Algorithm (VA) belongs to the maximum likelihood decoding category.

A Viterbi decoder drops the least likely Trellis path at each transmission stage. In this way it decreases decoding complexity with early rejection of unlike paths and gets its efficiency via concentrating on survival paths of the Trellis.

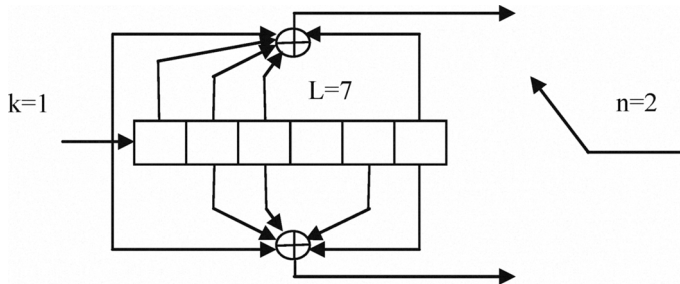


Figure 1: The Block diagram of a convolutional encoder.

There are hard decision and soft decision Viterbi decoders. A hard decision Viterbi decoder quantizes the binary analog signal using a 1-bit quantizer, and uses Hamming distance as a metric. A soft decision Viterbi quantizes a binary analog signal with multi-bit, uses Euclidean distance for a metric. Since the soft decision supplies more information than the hard decision, normally it has a better performance by 2-3 db. Several implementations of Viterbi decoders are reported in [5-12].

III. DESIGN OF THE VITERBI DECODER

A. Convolutional Encoder

A convolutional encoder can be viewed as a finite state machine. It can be implemented with registers, XOR gates, and a multiplexer. Figure 2 shows the encoder used in this design.

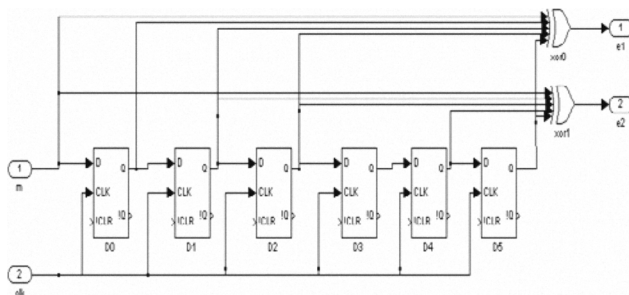


Figure 2: The convolutional encoder (2, 1, 7).

The two generator polynomials of the above encoder are listed as follows:

$$G_1(D) = 1 + D + D^2 + D^3 + D^6$$

$$G_2(D) = 1 + D^2 + D^3 + D^5 + D^6$$

As shown in the above figure, the encoder has a constraint length 7 and a code rate $\frac{1}{2}$. Some references suggest that, if the code rate is $\frac{1}{2}$, then a typical trace back depth is about five times to eight times of the constraint length of codes. Here we choose the trace back depth as 41 [13]. So the encoder starts encoding 35-bit message data, plus 6-bit 0s tail, from state 0. And it finishes at state 0 at the end of each message frame.

The Trellis diagram of the convolutional encoder has 64 states, and 41 stages. The encoder starts at state 0 at stage 0. After encoding all the k bits of information it ends up at state 0 again at stage 41.

B. Design of the Viterbi Decoder

The design of Viterbi decoder presented here is totally based on an ASM flowchart of Viterbi algorithm shown in Figure 3. As shown in the flowchart, we can see

1. Right after resetting, the decoder clears the counter and all registers to 0s, and starts to find the surviving predecessor states, by looking up Hamming distances, and comparing them, and saving the previous states from which we get smaller metric, for each state once a new symbol received.
2. The counter is increasing by 1. And generate surviving predecessor states as mentioned in 1.
3. If the counter does not reach to 40? Go to 2. Otherwise go to 4.
4. Again clear the counter to 0, generate surviving predecessor states as mentioned in 1, generate survivor states, and reconstruct the original message.
5. The counter is increasing by 1, generate surviving predecessor states as mentioned in 1, generate survivor states, and reconstruct the original message.
6. If the counter reaches 40, go to 4. Otherwise go to 5.

The first two steps process first 41 symbols. From step 4 the decoder starts to recreate original message, and stays in the last three steps. The whole process of Viterbi decoding is controlled by the ASM. To have a high speed, lookup tables and parallelism are used extensively. As we can see from the flow chart, the decoder has a latency of 41 clock cycles.

Based on the flowchart, the RTL code for the decoder is written in SystemVerilog. The whole RTL code includes one package and five other modules. The Trellis diagram and Hamming distance finding are implemented by lookup tables saved as functions in the package. The modules accomplish the rest part of Viterbi algorithm. Figure 4 shows the block diagram of the design. The block diagram has 4 inputs and 1 output. Block counter does the counting; Block s2p converts serial message into 2-bit output; Block aempst does surviving

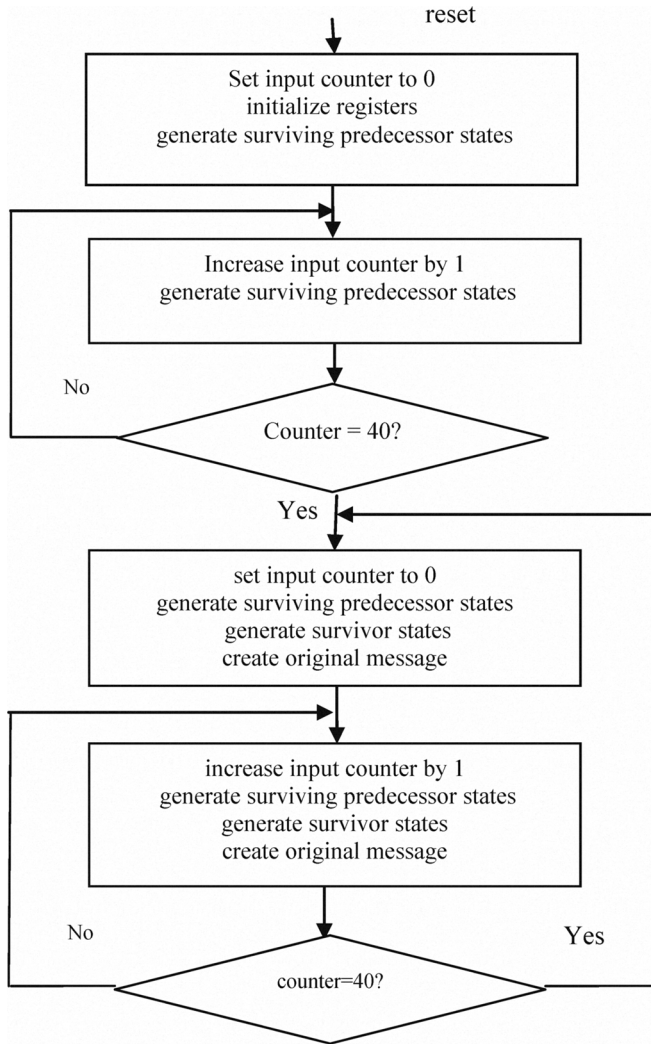


Figure 3: The flow chart of the ASM

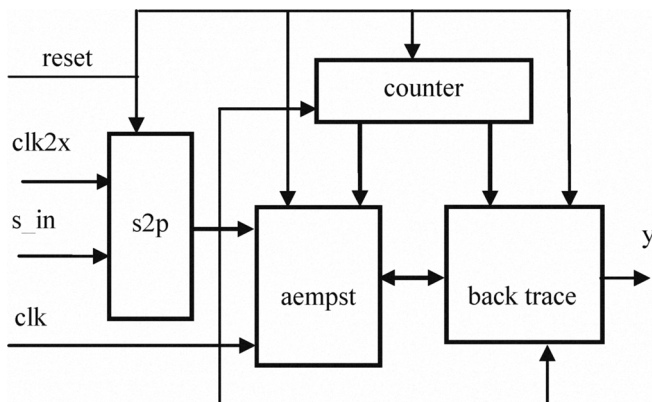


Figure 4: The block diagram of the Viterbi decoder

predecessor states and survivor state table generation; the back trace block recreates original message.

IV. FUNCTIONAL VERIFICATION OF THE VITERBI DECODER DESIGN

The functional verification of a RTL design is always a major challenge, since, first, we do not know when and if we have verified at the design sufficiently, and second, we need ways to check the correctness of the simulation results quickly and efficiently. For the first issue, ideally we hope our stimuli have touched all possible test states of the design. However, unfortunately, even a small design would have more test states than anyone can verify. Therefore, only limited test states will be verified for every design. To gauge the quality of test cases and what design features have been tested, the functional coverage, code coverage, and assertion checker are used during verification. To tackle the second issue, a test environment with self-checking capability is often used. In such kind of test environment, all the simulation results are checked for correctness in real time. However, to develop such a testing environment is time consuming. Figure 5 demonstrates a simplified self-checking test environment. In the figure the driver is basically stimuli generator, and it feeds them to both the reference model and the Design Under Test (DUT). From the stimuli, the reference model creates expected values to be used for comparison with the simulation results. This model could be written in high level language, like C/C++, Matlab/Simulink, SystemC, or Hard Description Language (HDL), like Verilog, VHDL, or Hard Description and Verification Language (HDVL), like SystemVerilog. The DUT is just a design in RTL code. The checker verifies the results against expected values during simulation.

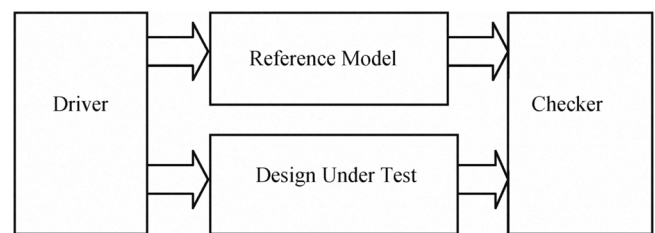


Figure 5: A self-checking verification environment.

Our design is verified in two ways independently. One uses self-checking test bench written with SystemVerilog. The other uses Simulink/ModelSim co-simulation.

A. Co-simulation with Simulink and Modelsim

First, the design is verified by using co-simulation with Simulink and Modelsim. Simulink has an interface link for ModelSim. The interface provides a bidirectional link

between ModelSim and Simulink, and so it gives us an integrated test environment to co-verify and co-simulate our design functionally.

In the Simulink co-simulation environment, we reuse our Simulink design as reference model just like the one shown in Figure 5. Figure 6 shows the block diagram of the co-simulation. On the top of the diagram, is a communication system consisting of convolutional encoder, Binary Phase-Shift Keying (BPSK) modulator, AWGN channel, BPSK demodulator, and a Simulink Viterbi decoder model. The reset signal, clock signal generator, the RTL design, and scopes are on the bottom. In order to facilitate comparing the waveforms, some delay units were added to the co-simulation environment. As shown in the diagram, both the Simulink Viterbi decoder model and Viterbi Decoder in RTL are basically fed by the same inputs. The only difference here is that since the RTL model cannot accept framed-based inputs, a converter to un-buffer the data block is inserted.

Figure 7 is a capture of co-simulation results. The first waveform from the top is the clock signal. Below the clock is the message sent. The waveform in the third row is the output of the Simulink Viterbi decoder model, and the bottom one the output of the Viterbi Decoder in RTL. As we can see that the waveforms from the Simulink model and the RTL design are perfectly matched.

B. Verification with a Self-checking Testbench

We used SystemVerilog creating a self-checking verification environment as illustrated in Figure 5. Defined by SystemVerilog classes, the driver can generate both directed and random stimuli. By randomizing the class, constrained Random Test (CRT) cases are obtained. Another class is used to mimic a communication channel. If needed, AWGN can be randomly injected into stimuli. Since from the stimuli, the output of Viterbi decoder can be predicted, the main function of the reference model is to save stimuli and send them out after 41 clock cycle delay for comparison. The checker compares the outputs of Viterbi decoder against the ones of the reference model. If mismatch found, the checker issues warnings. The testbench can also generate coverage reports to show the progress of the verification.

We verified the design under two scenarios: without AWGN and with AWGN. Once we verified it is working under no AWGN scenario. We started to inject errors to the stimuli to mimic AWGN. The results from enormous test cases show that if less than 5 out of 41 symbols are corrupted, the decoder can correctly reconstruct the original message. Figure 8 shows a simulation waveform capture. As shown in the figure from top to bottom, the first signal in the top is the *reset*, followed by *clock* signal. The third signal *m* is the information bit stream randomly generated by the test driver. The fourth signal *x* is from encoding *m*. The fifth signal *r* is the received signal at the decoder. There are some difference

between *m* and *r* due to AWGN. The sixth one *md* is the delayed *m*, and is used for visual comparison here. The last signal *y* is the output of the decoder. As we can see that decoder correctly reconstructs the original information bits, since *md* and *y* are exactly same.

V. FPGA IMPLEMENTATION

The Altera Quartus II is a comprehensive FPGA design environment. We use it to perform the logic synthesis and fitting for our design. Its TimeQuest timing analyzer is used for static timing analysis. Before the compilation, the design was constrained with a SDC file, which specifies clock frequency, input and output delays, timing exceptions, and other timing requirements. Table I lists some of its FPGA implementation results from Quartus II.

VI. CONCLUSION

Design and FPGA implementation of a high-speed hard-decision Viterbi decoder with a constraint length of 7 and a code rate of $\frac{1}{2}$ is presented in this paper as a case study using SystemVerilog and co-simulation. The design was functionally verified in two different self-checking simulation environments. First the RTL design was co-simulated with Simulink and ModelSim by using its Simulink model as a reference model. Then the RTL code was verified in a self-checking verification environment developed with SystemVerilog. In this case CRT is used for verifying the design, and functional coverage was used for checking the verification progress. The RTL code was verified in numerous test cases, including several directed test cases. The synthesis results show that the FPGA implementation can run as high as 510 MHz. It has a throughput of 510 Mbps and a latency of 80.39 ns. The proposed design is suitable for high speed communications without compromising on performance. The study is also regarded as a learning example for using SystemVerilog and Modelsim-Simulink co-simulation in digital system design and verification.

TABLE I. IMPLEMENTATION RESULTS

Family	Stratix III
Device	EP3SL50F484C2
Combinational ALUT	13832
Register	8484
Clock Speed	510 Mhz
Latency	80.39 ns
Throughput	510 Mbps

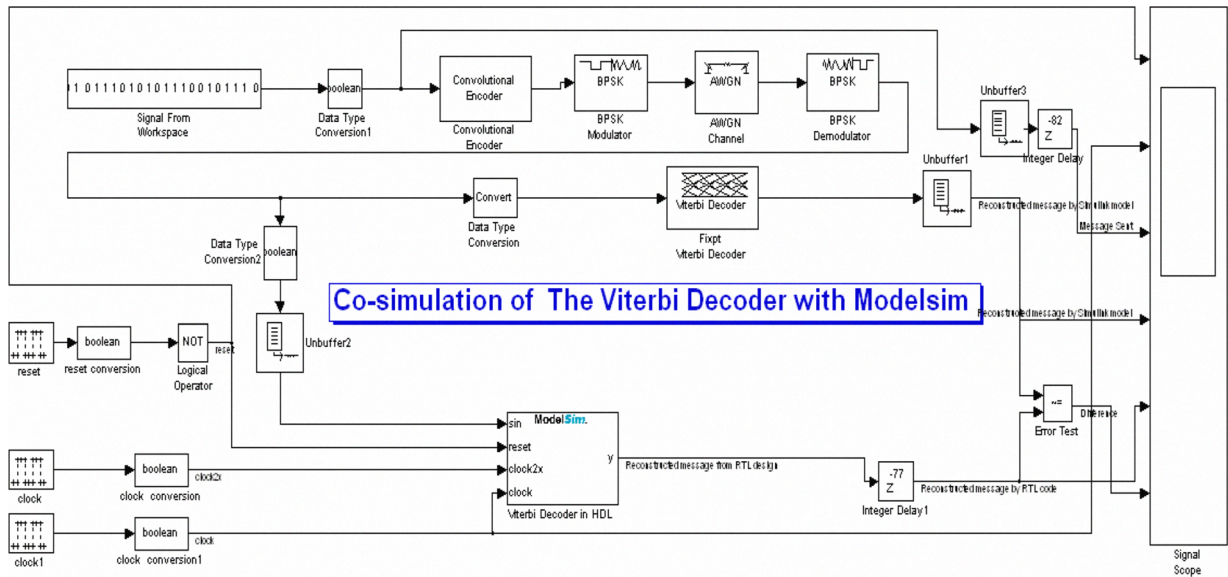


Figure 6: Co-simulation diagram of the Viterbi decoder.

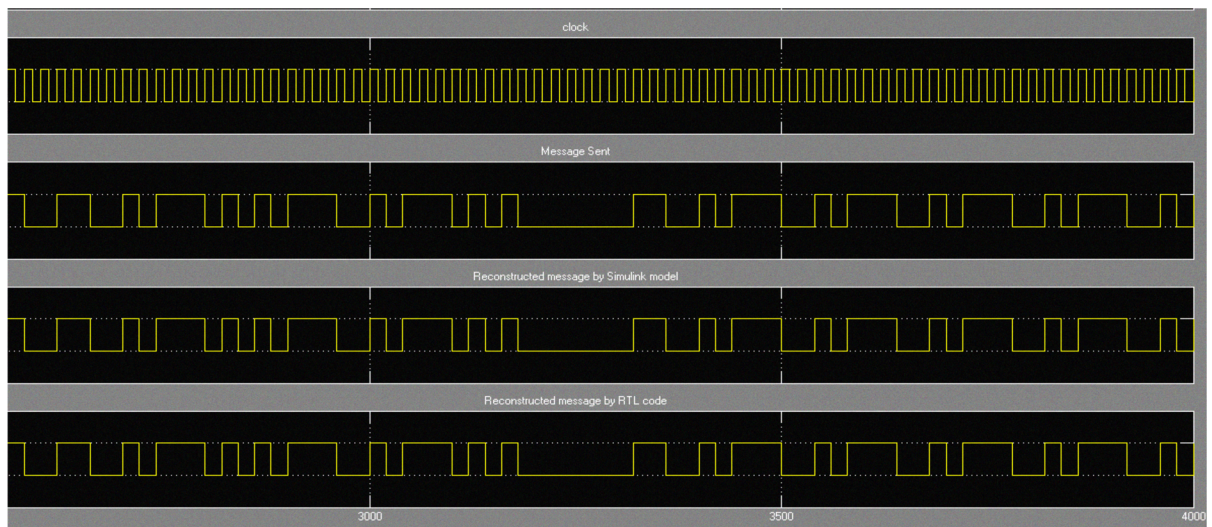


Figure 7: The waveforms from co-simulation.

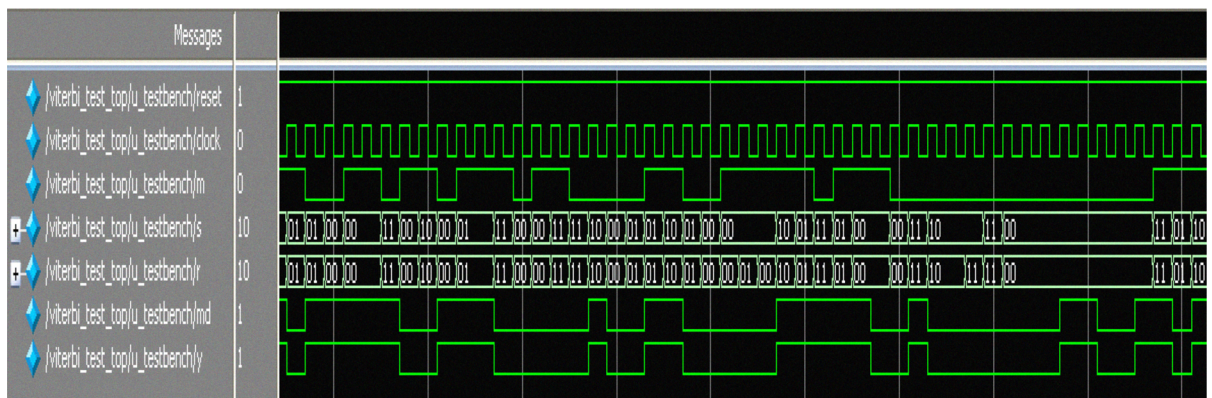


Figure 8: A simulation waveform when stimuli are random.

REFERENCES

- [1] A. J. Viterbi, "Error Bounds for Convolutional Codes and an Asymptotically Optimum Decoding Algorithm," *IEEE Trans. Inform. Theory*, vol. IT-13, pp. 260-269, Apr. 1967.
- [2] Man Guo, M. Omair Ahmad, M. N. S. Swamy, Chunyan Wang, "FPGA Design and Implementation of a Low-Power Systolic Array-Based Adaptive Viterbi Decoder," *IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS*, VOL. 52, NO. 2, pp. 350 -365, FEBRUARY 2005.
- [3] Hao Yan, Xiao-Lang Yan, "Design and Implementation of a High-speed and Area-efficient Viterbi Decoder", *IEEE*, 2006.
- [4] John G. Proakis, "Digital Communications," 4/e, McGraw-Hill, 2000.
- [5] Meilana Siswantol, Masuri Othman, Edmond Zahedi, "VLSI Implementation of 1/2 Viterbi Decoder for IEEE P802.15-3a UWB Communication," *ICSE2006 Proc.*, pp. 666-670, 2006, Kuala Lumpur, Malaysia.
- [6] Sridhar Nandula, Yepuri Sudhakara Rao, and Siva Prasad Embanath, "High speed area efficient configurable Viterbi decoder for WiFi and WiMAX systems," *International Conference on Intelligent and Advanced Systems*, pp. 1396 – 1399, 2007.
- [7] Kelvin Yi-Tse Lai, "An Efficient Metric Normalization Architecture for High-speed Low-power Viterbi Decoder," *IEEE* 2007.
- [8] C. Arun, V. Rajamani, "Design and VLSI implementation of a Low Probability of Error Viterbi decoder", *First International Conference on Emerging Trends in Engineering and Technology*, DOI 10.1109/ICETET.2008.180, pp. 418 – 423, *IEEE* 2008.
- [9] A. R. Abdul Shakoor, V. Szwarc, and T. A. Kwasniewski, "High Speed Viterbi Decoder for W-LAN and Broadband Applications", *regular session B: digital system design*, pp 25 – 28, *IEEE* 2004.
- [10] Yiqun Zhu, Mohammed Benaissa, "Reconfigurable Viterbi Decoding Using a New ACS Pipelining Technique," *Proceedings of the Application-Specific Systems, Architectures, and Processors, ASAP'03*.
- [11] Chao Cheng, Keshab K. Parhi, "Hardware Efficient Low-Latency Architecture for High Throughput Rate Viterbi Decoders," *IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—II: EXPRESS BRIEFS*, VOL. 55, NO. 12, pp 1254 – 1258, DECEMBER 2008.
- [12] QIN Xiang-Ju, ZHU Mmg-Cheng, WEI Zhong-Yi, CHAO Du, "An Adaptive Viterbi Decoder Based on FPGA Dynamic Reconfiguration Technology", pp. 315-318, *ICFPT* 2004.
- [13] C. Fleming, *A Tutorial on Convolutional Coding with Viterbi Decoding*, 2006, <http://home.netcom.com/~chip.f/viterbi/tutorial.html>.